COBCheck: Article-Mapped GDPR Compliance Analysis for COBOL

Sai Krishna and Srikrishna | Target: FSE 2027 research track | Reissue, supersedes RISHA_COBCheck_COBSmell_Brief (April 2026)

Core Argument. Cross-program data flow in COBOL applications is now recoverable by shipped tooling, so tracing where personal data travels is no longer the open problem. What remains open is deciding whether a recovered flow constitutes a violation of a specific GDPR obligation, which requires mapping flow facts onto article-level requirements and evaluating that mapping against a ground truth. COBCheck claims that article mapping, rather than flow recovery, is where correctness in source-level compliance analysis is won or lost, and that an LLM applied directly to COBOL source can decide locally scoped obligations while failing on obligations whose evidence is distributed across programs.

The Six Questions

What is the problem?

Personal data in mainframe applications moves through copybook-declared structures, working storage, files and inter-program calls, and the obligations that govern such movement are stated in legal articles rather than in code. Existing source-level compliance analysis targets Android and Java, where sources and sinks are API calls with known privacy semantics. COBOL has no such API surface, so sources and sinks must be derived from data declarations and file usage rather than looked up. The problem is whether article-level GDPR obligations can be decided from COBOL source, and whether the deciding step can be evaluated separately from the flow recovery step. Stated as a failure condition, an analysis may recover a flow correctly and still attribute the wrong obligation to it, or attribute none.

Why is it important?

Mainframe applications in banking and insurance hold regulated personal data, and practitioner reporting states that the mainframe was not designed for consent management, data subject access requests, or field-level auditability [8]. Data relationships in such systems live in COBOL programs, copybooks and VSAM files rather than in formal schemas [8], so a compliance question cannot be answered from the database schema alone. Enforcement is not hypothetical. Cumulative GDPR penalties have reached the order of billions of euro since the regulation took effect [7]. The obligation therefore carries legal weight, but the artifact a controller would have to inspect to demonstrate compliance is source code that few current tools reason about in legal terms.

What are the existing solutions, and why are they not enough?

Ferrara and Spoto [1] and Ferrara, Olivieri and Spoto [2] tailor taint analysis to GDPR by starting from the controller's declared policy, applying taint analysis to detect leakages, and reconstructing the flow to check whether the flow is permitted. The strength usable here is the two-stage separation of detection from permission checking, which is the separation COBCheck adopts. The weakness is that the policy is assumed given and the setting is object-oriented code with known sinks, neither of which holds for COBOL. Static taint analysis for Android, including FlowDroid, IccTA and Amandroid [6], tracks flows from defined sources to defined sinks with high precision. The strength is the evaluation discipline of the area. The weakness for our purpose is stated within the area itself, that bare taint analysis detects the presence of leaks but cannot answer how data is manipulated or how it is protected, which is what compliance assessment requires [4]. GDPR-Bench-Android [6] constructs a benchmark for automated GDPR compliance detection and positions rule-based analysis, LLM-based code analysis and agentic or retrieval-augmented methods against it. The strength is the benchmark design, which our evaluation should follow rather than reinvent. The weakness we address is the setting, since Android supplies API-level privacy semantics that COBOL does not, so a method that performs well there is not predicted to transfer. AWS Transform for mainframe, Data Analysis capability [5], recovers data relationships across programs, JCL jobs and datasets, and catalogs field-level metadata for copybook structures with the COBOL level hierarchy preserved. The demonstration corpus is CardDemo, which is also our evaluation corpus, and CardDemo is reported to contain 59 COBOL data structures across its copybooks [5]. The strength is that cross-program lineage may be treated as available infrastructure rather than as a contribution. The weakness is that lineage answers which programs touch which data, and stops short of stating which obligation a given flow engages. Micro Focus Enterprise Analyzer [9] maintains a metadata repository over control flow, data flow, file and database interactions, with impact analysis, business rule extraction and code slicing. The strength is maturity of cross-system dependency recovery on COBOL. The weakness is identical in kind, since none of the reported capabilities attaches a legal obligation to a recovered dependency. The gap that survives is narrow and specific. No published work, academic or commercial, decides GDPR article-level obligations from COBOL source and evaluates that decision against a ground truth.

What is the proposed idea?

Input. COBOL programs, copybooks and JCL from a target application, together with an article-level obligation set drawn from GDPR. Process. Personal data elements are identified from data declarations rather than from API signatures. Flow facts are recovered over the shared code property graph infrastructure, intra-program and across program boundaries. Each recovered flow is then mapped onto the obligations it engages, producing an article attribution with the source evidence attached. The mapping instrument is the object of study. Output. Article-attributed findings, each carrying the program locations, the data path and the obligation engaged. What is different. The increment holds the problem fixed, which is source-level GDPR compliance checking, and varies the language and the deciding layer. Prior work varies the analysis technique while assuming the obligation mapping is trivial. COBCheck treats the mapping as the contribution, and reports what the community does not currently know, which is how much of compliance decision error is attributable to mapping rather than to flow recovery, and whether that split differs between locally scoped and cross-program obligations.

Why should we believe it?

Evaluation follows a constructed benchmark design with injected violations over AWS CardDemo and IBM GenApp, with a two-rater oracle. Injection gives ground truth that mined corpora cannot supply for COBOL, since no issue and patch record exists for compliance defects in this setting. Three arms are compared. An LLM applied to COBOL source without recovered flow facts, a flow-recovery arm without article mapping, and the full pipeline. The comparison isolates the claim, since the differential between locally scoped and cross-program obligations is the result being sought rather than an aggregate score. Assumptions to state explicitly. Injected violations may not reflect the distribution of naturally occurring ones. The obligation set is a selection from GDPR rather than the whole regulation. Article attribution is a legal interpretation, and the oracle records the raters' interpretation rather than an adjudicated answer.

Where are the artifacts?

Documented source, a README following the lab checklist, the injected-violation benchmark with the injection scripts, the obligation mapping instrument, and the rater instructions. Under double blind, the artifact is released through an anonymized archive, with the project page held until acceptance.

What This Stands On

Field	Detail
Base paper	Ferrara, P., Olivieri, L., Spoto, F. Tailoring Taint Analysis to GDPR. Privacy Technologies and Policy, Springer, 2018. Relationship: port. The sentence stood on is that, starting from the policy established by the controller of sensitive data, taint analysis detects potential leakages and the flow is then reconstructed to check whether the flow is allowed. The port is to a language where sources, sinks and the policy binding are not given, so the result is not predictable from the original setting.
Theoretical basis	The General Data Protection Regulation as a legal instrument, taken at article level. The obligation set is derived from named articles rather than from an intuition about privacy, which is what allows a mapping to be right or wrong. The DPDP Act follows as a second instrument once GDPR is settled, per the scope decision of 30 August.
Empirical standards	Engineering Research, Benchmarking and Simulation, Sampling supplement

Mapping to the SIGSOFT Empirical Standards

Standard	Why it applies	Where the work currently falls short
Engineering Research	COBCheck is a technique and tool evaluated on a constructed task	No stated baseline for the flow-recovery arm, since shipped lineage tooling has not been run on the corpus for comparison
Benchmarking and Simulation	Evidence comes from scoring arms against an injected-violation benchmark	Benchmark construction protocol not documented, injection distribution not justified, and no held-out portion defined
Sampling supplement	Programs, data elements and injected violations are all sampled	Sampling frame for which programs and which obligations are covered is not defined

Not applicable, despite appearances. Experiments with Human Participants does not apply, because the two raters construct the oracle and are not study subjects. Repository Mining does not apply, because the corpus is two fixed applications rather than a mined population. Case Study does not apply, because neither application is studied as a setting in its own right. Qualitative Surveys and Grounded Theory do not apply, because the obligation coding scheme is fixed in advance from the regulation rather than developed from the data.

How It Will Be Evaluated

Metric	Measures	Why this construct	Source	What counts as good
Precision, recall and F1 per obligation	Detection quality against injected ground truth	The standard reporting form for compliance detection benchmarks	GDPR-Bench-Android [6]	Reported per article rather than pooled, since pooling hides the claim
Cross-program against intra-program recall	Whether performance splits by evidence locality	The claim is about the split rather than the absolute level	New to this work, argued from the problem statement	LLM-only arm materially lower on cross-program than on intra-program, with the full pipeline closing the difference
Article attribution accuracy	Whether the correct obligation is named for a correctly recovered flow	Isolates mapping error from recovery error, which is the core claim	New to this work, argued from the core claim	Reported separately from detection, so that a mapping failure is visible as such
Inter-rater agreement on the oracle	Reliability of the ground truth	An interpretive oracle without an agreement figure cannot carry a result	Standard practice for annotated benchmarks	Threshold fixed and agreed before annotation begins
False positives on unmodified programs	Whether the analysis fires on clean code	A compliance tool that over-reports is unusable in audit	Engineering Research standard	Reported on the uninjected baseline copies of both applications

State of the Art and References

Checked in this pass. Academic work on COBOL and GDPR at source level, which returned nothing. Code-level GDPR compliance analysis, which is concentrated in Android and Java [1, 2, 3, 4, 6]. Recent benchmark work on automated GDPR compliance detection [6]. Commercial and cloud mainframe analysis capabilities [5, 9]. Patent coverage on COBOL data lineage and on sensitive data discovery through application data flows. What changed since the April brief. The evaluation corpus moved to CardDemo and GenApp with injected violations. The DPDP framing moved after the Rules were notified, and GDPR now comes first. Most importantly, cross-program lineage recovery on COBOL is now shipped and was demonstrated on CardDemo, so the argument that a database-level tool cannot see the paragraph copying a customer record into an extract file no longer separates the work from the field. The novelty moved to article mapping. What the student must check again. Whether any 2026 work maps recovered flows to legal obligations in any language, since a positive finding there weakens the claim regardless of the language. Forward citations of the base paper. The requirements engineering and legal informatics venues, where compliance work appears under different names. Whether GDPR-Bench-Android has a published version superseding the preprint. If the results came back flat, the design tells us which kind of flat. Equal performance across arms with high absolute scores would mean the obligations chosen are locally decidable and the corpus is too easy. Equal performance with low scores would mean the mapping instrument is not yet usable. Both are findings, and both are distinguishable from failure.

Technical Tasks

Task	What to produce	Done when
Profile both applications	Program, copybook and JCL counts, personal data element inventory, cross-program call and file dependency map	Inventory reproduces from a script over the checked-out corpus
Complete or document the cross-program flow component	A working component, or a written statement of what is unimplemented	The gate research task is answered from the code rather than from recall
Implement the article mapping layer over recovered flows	Mapping module taking flow facts and emitting article attributions with evidence	Runs end to end on one seeded violation of each type
Build the injected-violation benchmark	Injected corpus copies, injection manifest, uninjected control copies	Manifest and corpus regenerate from the scripts
Implement the LLM-only arm	Baseline runner applying an LLM to source without recovered flow facts	Produces article attributions in the same output format as the full pipeline
Run a shipped lineage tool over the corpus for the recovery baseline	Recovered lineage in a comparable format, or a documented statement of why the tool could not be run	Comparison is possible, or the impossibility is recorded for threats to validity
Score all arms	Per-article and per-locality results tables	Every metric in the evaluation table is populated

Research Tasks

1. GATE. Establish how much of the cross-program PII flow engine executes on CardDemo today, and how much remains a design specification. Record the answer as a component-level status. If the engine does not run, stop and return to Ch, because the paper's arms cannot be separated without it.
2. Define the obligation set. Name the GDPR articles in scope, and for each, state the code-level condition that would evidence a violation. Articles out of scope are listed with the reason.
3. Define personal data identification for COBOL. State how a data element is judged personal from its declaration, name and usage, since no API-level signal exists.
4. Write the injection protocol. State violation types, how many of each, where they may be placed, and how intra-program and cross-program cases are balanced.
5. Fix the oracle procedure. Rater instructions, the agreement statistic, the threshold, and the disagreement resolution rule, all settled before annotation starts.
6. Run the deeper literature pass and produce the review spreadsheet, with the strength and weakness columns filled for every closest work.
7. Draft the related work differentiation against AWS Transform and Enterprise Analyzer explicitly. Omitting either is a visible gap, and CardDemo is the AWS demonstration corpus.

Ordering and Dependencies

The gate research task on the flow engine gates every technical task. The obligation set and the personal data definition gate the mapping layer, since obligations that are not defined cannot be attributed. The injection protocol gates the benchmark build. The oracle procedure gates annotation, and annotation gates all scoring. The corpus profile is placed early because a late profile would invalidate the injection balance already built.

Decisions for Ch

1. Whether to proceed if the flow engine turns out to be a design specification, given that the article-mapping contribution alone may not carry a research track paper.
2. Venue confirmation, given the remaining work against the FSE 2027 date.
3. Whether the provisional filing is made before any public disclosure, including preprint posting.
4. Whether DPDP enters this paper or stays in the second paper.

References

1. Ferrara, P., Spoto, F. Static Analysis for GDPR Compliance. ITASEC, 2018.
2. Ferrara, P., Olivieri, L., Spoto, F. Tailoring Taint Analysis to GDPR. Privacy Technologies and Policy, Springer, 2018.
3. Hjerppe, K., Ruohonen, J., Leppanen, V. Annotation-Based Static Analysis for Personal Data Protection. IFIP Summer School, Springer, 2020.
4. Toward an Android Static Analysis Approach for Data Protection. arXiv:2402.07889, 2024.
5. AWS. Data Analysis with AWS Transform for mainframe: Understand your applications data landscape. AWS re:Post, 2026.
6. GDPR-Bench-Android: A Benchmark for Evaluating Automated GDPR Compliance Detection in Android. arXiv:2511.00619.
7. Secure Privacy. GDPR Compliance in 2026: The Complete Guide.
8. Archon. COBOL Modernization and Migration: A Complete Enterprise Guide, 2026.
9. IN-COM Data Systems. Top COBOL Static Code Analysis Solutions for Mission-Critical Systems, 2025.
10. Note. Entries 5, 7, 8 and 9 are practitioner sources used for motivation and for the commercial baseline argument. They are to be replaced or supplemented with citable sources in the paper, and the arXiv entries checked for published versions before submission.

AI Disclosure

The original ideas, analysis, and research synthesis in this report are the intellectual contribution of the authors. AI-assisted tools were used to enhance the clarity and presentation of the writing.